Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-212БВ

Студент: Штыков М.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 28.10.2026

Москва, 2026

Постановка задачи

Вариант 1. Суммирование строк через разделяемую память.

Разработать две программы на C99 для Linux. Родитель получает имя файла и строки целых чисел типа int. Дочерний процесс вычисляет сумму каждой строки и записывает результат в файл. Обмен организовать через именованный объект общей памяти и mmap, синхронизацию — через именованные семафоры. Имена объектов должны различаться для разных экземпляров программ. Необходимо обрабатывать системные ошибки.

Общий метод и алгоритм решения

1. server читает имя файла, формирует имена IPC из PID и времени запуска. Создаёт общую память с O_CREAT | O_EXCL, задаёт размер и отображает её с MAP_SHARED. Два семафора request и reply создаются со значением 0.

2. fork создаёт дочерний процесс. В дочерней ветке execv запускает отдельный client и передаёт ему имя файла и имена IPC. Ребёнок подключается к памяти и семафорам, открывает файл и подтверждает готовность.

3. Родитель записывает блок текста в память и уведомляет ребёнка через request. Ребёнок разбирает числа, записывает суммы завершённых строк и подтверждает обработку через reply. Только затем родитель выводит эхо и передаёт следующий блок.

4. Пустая строка или EOF завершают ввод. Родитель передаёт маркер UINT32_MAX. Ребёнок обрабатывает последнюю строку, закрывает файл и завершается. Родитель проверяет статус через waitpid и освобождает ресурсы.

Использованные системные интерфейсы

shm_open, ftruncate, mmap — создание и отображение общей памяти; sem_open, sem_post, sem_timedwait — создание семафоров, уведомление и ожидание; fork, execv, waitpid — управление процессами; open, read, write, close — ввод-вывод; munmap, shm_unlink, sem_close, sem_unlink — освобождение ресурсов. readlink находит client рядом с server, clock_gettime участвует в формировании имён и сроков ожидания.

Особенности реализации

Состояние Parser сохраняется между блоками по 4096 байтов, поэтому длина строки и количество чисел не фиксированы. Проверяются диапазон int и переполнение суммы long long. Ожидание семафора выполняется интервалами по секунде с проверкой другого процесса. После подключения ребёнка имена IPC удаляются, но открытые объекты продолжают работать до закрытия ссылок. stdio.h не используется.

Код программы

Структура проекта: server.c, client.c и общий заголовок ipc.h. Ниже приведены ключевые фрагменты; полный код и обработка ошибок находятся в исходных файлах.

server.c — создание процесса и запуск client

```
child = fork();
if (child < 0) { report_error("fork"); goto cleanup; }
if (child == 0) {
   char *const args[] = {"client", filename, shm_name,
                         request_name, reply_name, NULL};
   execv(client_path, args);
   report_error("execv client");
   _exit(1);
}
```

server.c — передача блока в функции exchange

```
shared->length = length;
if (sem_post(request) < 0) {
   report_error("sem_post request"); return -1;
}
if (wait_reply(reply, child) < 0) {
   report_error("wait reply"); return -1;
}
```

После подтверждения проверяется shared->status; при успехе родитель выводит эхо блока. Новая запись разрешена только после ответа ребёнка.

client.c — разбор блока и подтверждение

```
for (uint32_t i = 0; i < shared->length; ++i) {
   if (shared->data[i] == '\n') {
      if (save_sum(file, &parser) < 0) {
         failed = 1; break;
      }
   } else consume(&parser, shared->data[i]);
}
shared->status = failed;
if (sem_post(reply) < 0) {
   report_error("sem_post reply"); goto cleanup;
}
```

До обработки client ожидает request. consume накапливает цифры и завершает числа по разделителям; save_sum проверяет команду, записывает сумму через write и сбрасывает Parser. Маркер завершения обрабатывается отдельной веткой.

ipc.h — формат обмена

```
typedef struct {
   uint32_t length;
   int status;
   char data[4096];
} Shared;
```

Протокол работы программы

Сборка и проверка выполнены на Ubuntu 24.04.3 LTS, стандарт C99. Исполняемые server и client располагаются в одном каталоге.

```
cc -std=c99 -Wall -Wextra -Wpedantic server.c -o server -pthread -lrt
cc -std=c99 -Wall -Wextra -Wpedantic client.c -o client -pthread -lrt
./server
```

Тест 1. Несколько строк и завершение пустой строкой

```
Output filename: result.txt
Enter integers, one command per line. Empty line = finish.
1 23
1 23
sshty@Sht:/mnt/c/Users/sshty/Desktop/OS 2026/lab3$ cat result.txt
24
```

Тест 2. Границы int и сумма long long

```
sshty@Sht:/mnt/c/Users/sshty/Desktop/OS 2026/lab3$ ./server
Output filename: result.txt
Enter integers, one command per line. Empty line = finish.
-2147483648 2147483647 1
-2147483648 2147483647 1

sshty@Sht:/mnt/c/Users/sshty/Desktop/OS 2026/lab3$ cat result.txt
0
```

Тест 3. Ошибочный ввод

Команды 12 abc, 1.5 2, +, 2147483648 и −2147483649 завершились с ненулевым кодом. Сумма ошибочной строки не записана. Сохранённые результаты предыдущих строк остались в файле.

```
Invalid integer, empty command or sum overflow.
```

Тест 4. Ошибки ресурсов

Проверены несуществующий каталог выходного файла, ошибка записи в /dev/full и отсутствие исполняемого client. Во всех случаях родитель завершился с ошибкой без зависания. Всего test.py выполнил 23 успешные проверки; сборка с -Werror прошла без предупреждений.

Вывод

Реализовано вычисление сумм строк в отдельном процессе с обменом через именованную разделяемую память. Два семафора обеспечивают очередность чтения и записи одного буфера. Учтены разрыв чисел между блоками, неверный ввод и ошибки системных операций. Проверки подтвердили корректность результатов и независимость параллельных экземпляров.